



**UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH**

**Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa**

Bachelor's Degree in Aerospace Vehicle Engineering

MPI-Based Heat Solver: A parallel implementation of the Jacobi method to solve heat distribution in a 1D domain

Mathematics and Computing Engineering
Marina Liao Colom Escolà

June 4, 2026

Abstract

This project presents the implementation of a parallel solver for the steady-state one-dimensional heat conduction equation in a composite wall. The numerical solution is obtained through a finite-volume discretization and an iterative Jacobi solver. Parallelization is achieved using the Message Passing Interface (MPI), where the computational domain is decomposed among multiple processes. Halo-node communication is employed to exchange boundary information between neighboring subdomains. The implementation demonstrates the effectiveness of distributed-memory parallelization for iterative solvers while highlighting the impact of communication overhead on scalability.

Contents

1	Introduction and Objectives	4
2	Problem Statement	5
3	Theoretical Background	6
3.1	Heat Conduction Equation	6
3.2	Finite Volume Method	6
3.3	Harmonic Averaging of Thermal Conductivity	6
3.4	Iterative Solution Methods	7
3.5	Parallel Computing with MPI	7
3.6	Domain Decomposition	7
4	Mathematical Formulation	8
5	Numerical Methodology	9
5.1	Mesh Generation	9
5.2	Initialization	9
5.3	Jacobi Iteration	9
5.4	Convergence Criterion	9
6	Parallel Implementation	10
6.1	Work Distribution	10
6.2	Local Data Structures	10
6.3	Halo Nodes	10
6.4	Halo Exchange	11
6.5	Collective Communications	11
7	Code Structure	12
7.1	Main Program	12
7.2	Mesh Generation Module	12
7.3	Pre-Solver Module	12
7.4	Jacobi Solver Module	12
7.5	Output Module	13
7.6	Testing Framework	13
8	Verification and Testing	14
8.1	Mesh Verification	14
8.2	Jacobi Solver Verification	14
8.3	Parallel Verification	14
9	Performance Analysis and Scalability	15
9.1	Execution Time	15
9.2	Speedup	15
9.3	Parallel Efficiency	16
9.4	Communication Overhead	16
10	Conclusions	17
10.1	Numerical Results	17
10.2	Performance Analysis	17
10.3	Limitations	17
10.4	Final Remarks	18
11	Bibliography	19

A	main.c script	20
B	mesh_generator.c script	23
C	pre_solver.c script	24
D	solver_jacobi.c script	25
E	printing.c script	27
F	test_mesh.c script	29
G	test_jacobi.c script	30
H	Headers files used for the project	31

1 Introduction and Objectives

The numerical solution of partial differential equations (PDEs) plays a fundamental role in modern engineering and scientific computing. Many physical phenomena, including fluid flow, structural deformation, electromagnetism, and heat transfer, can be described through differential equations whose analytical solutions are often unavailable for realistic geometries and boundary conditions. Consequently, numerical methods have become essential tools for obtaining approximate solutions with sufficient accuracy.

As computational models become increasingly complex and high-resolution simulations are required, the computational cost associated with solving large systems of equations grows significantly. Parallel computing techniques allow these problems to be distributed across multiple processors, reducing execution time and enabling simulations that would otherwise be computationally prohibitive. Among the available parallel programming paradigms, the Message Passing Interface (MPI) [Message Passing Interface Forum, 2023] has become the standard for distributed-memory high-performance computing systems.

In this project, a parallel MPI-based solver is developed to compute the steady-state temperature distribution in a one-dimensional composite wall (Section 2). The governing heat conduction equation is discretized using a finite-volume approach, resulting in a sparse linear system that is solved iteratively using the Jacobi method. The computational domain is partitioned among multiple MPI processes through domain decomposition, and communication between neighboring subdomains is achieved using halo-node exchanges.

The main objectives of the project are:

- To formulate and discretize a one-dimensional heat conduction problem.
- To implement an iterative Jacobi solver for the resulting linear system.
- To parallelize the computation using MPI.
- To analyze the scalability and performance of the parallel implementation.
- To evaluate the impact of communication overhead on distributed-memory computations.

The project provides practical experience with both numerical methods for partial differential equations and parallel programming techniques commonly employed in scientific computing applications.

2 Problem Statement

The objective of the code is to determine the steady-state temperature distribution across a one-dimensional composite wall composed of two materials with different thermal conductivities (Figure 1). The wall is subjected to fixed temperature boundary conditions at both ends, resulting in heat transfer by conduction through the structure.

The physical domain consists of a wall of length

$$L = 5m,$$

with an interface located at

$$x_c = 2.8m.$$

The thermal conductivity is piecewise constant and defined as

$$\lambda(x) = \begin{cases} 120W/(mK), & x < x_c \\ 180W/(mK), & x \geq x_c \end{cases}$$

The boundary conditions impose fixed temperatures at the extremities of the wall:

$$T(0) = 100, K$$

and

$$T(L) = 200, K.$$

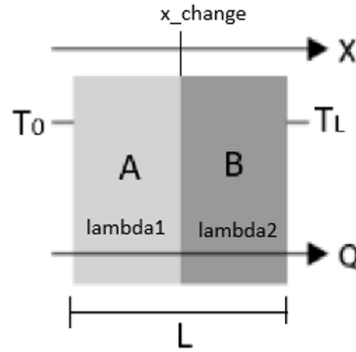


Figure 1: Problem sketch

Assuming one-dimensional steady-state heat conduction with no internal heat generation, the governing equation is

$$\frac{d}{dx} \left(\lambda \frac{dT}{dx} \right) = 0.$$

The analytical solution of this problem can be obtained for simple cases; however, the objective of this project is to solve it numerically using a finite-volume discretization and an iterative Jacobi solver [Saad, 2003]. The computational domain is discretized into a mesh containing a large number of nodes, leading to a system of linear algebraic equations.

To reduce computational time and demonstrate the use of distributed-memory parallelism, the domain is partitioned among several MPI processes. Each process is responsible for solving a local portion of the mesh while exchanging boundary information with neighboring processes through halo-node communications. .

3 Theoretical Background

3.1 Heat Conduction Equation

Heat transfer by conduction occurs due to temperature gradients within a material [Incropera et al., 2007]. According to Fourier's law, the heat flux is proportional to the negative temperature gradient:

$$q = -\lambda \frac{dT}{dx}$$

where:

- q is the heat flux (W/m^2),
- T is the temperature (K),
- λ is the thermal conductivity ($W/(mK)$).

For a one-dimensional steady-state problem without internal heat generation, conservation of energy leads

$$\frac{d}{dx} \left(\lambda \frac{dT}{dx} \right) = 0.$$

When the thermal conductivity varies throughout the domain, as in a composite wall, this equation must be solved numerically. The continuity of heat flux at material interfaces must also be enforced to ensure physically meaningful results.

3.2 Finite Volume Method

The finite volume method (FVM) is a discretization technique based on the conservation laws governing the physical system. The computational domain is divided into a finite number of control volumes, and the governing equation is integrated over each volume [Versteeg and Malalasekera, 2007].

Applying the divergence theorem transforms the differential equation into a balance between fluxes entering and leaving the control volume. For a one-dimensional problem, the discretized equation can be expressed as

$$a_P T_P = a_W T_W + a_E T_E,$$

where T_P denotes the temperature at the current node and T_W and T_E correspond to neighboring nodes.

One of the main advantages of the finite volume method is that conservation is guaranteed at the discrete level, making it particularly suitable for heat transfer and fluid mechanics applications.

3.3 Harmonic Averaging of Thermal Conductivity

The heat conduction problem considered in this project (Section 2) involves two different materials with distinct thermal conductivities. At the interface between materials, the conductivity used in the discretization must accurately represent the thermal resistance of both media [Incropera et al., 2007].

The harmonic average is commonly employed:

$$\lambda_h = \frac{2\lambda_1\lambda_2}{\lambda_1 + \lambda_2}.$$

This formulation preserves continuity of heat flux across material interfaces and avoids inaccuracies that could arise from arithmetic averaging.

3.4 Iterative Solution Methods

The discretization process transforms the governing differential equation into a system of linear algebraic equations:

$$A\mathbf{x} = \mathbf{b}.$$

For large-scale problems, direct methods such as Gaussian elimination become computationally expensive in terms of both memory and execution time. Iterative methods provide an alternative approach by progressively improving an initial estimate of the solution.

The Jacobi method [Saad, 2003] is one of the simplest iterative solvers. The coefficient matrix is decomposed as

$$A = D + L + U,$$

where D contains the diagonal entries, while L and U contain the lower and upper triangular components.

The iterative update is given by

$$x^{(k+1)} = D^{-1} \left(b - (L + U)x^{(k)} \right).$$

Although the Jacobi method converges more slowly than advanced techniques such as Conjugate Gradient or Multigrid methods, its simplicity and inherent parallelism make it an excellent choice for educational purposes and distributed-memory implementations.

3.5 Parallel Computing with MPI

The Message Passing Interface (MPI) [Message Passing Interface Forum, 2023] is a standardized communication library designed for distributed-memory parallel systems. Each process executes its own copy of the program and communicates explicitly with other processes through message passing.

MPI provides:

- Point-to-point communication functions such as `MPI_Send()` and `MPI_Recv()`.
- Collective communication routines such as `MPI_Broadcast()`, `MPI_Gather()`, and `MPI_Allreduce()`.
- Process management and synchronization mechanisms.

Because each process owns only a portion of the computational domain, data located at subdomain boundaries must be exchanged periodically. This requirement introduces communication overhead, which becomes a critical factor in parallel performance.

3.6 Domain Decomposition

Domain decomposition is one of the most common strategies for parallelizing numerical simulations. The computational mesh is divided into several subdomains, each assigned to a different process.

For a one-dimensional mesh containing N nodes and P processes, each process receives approximately

$$N_i \approx \frac{N}{P}$$

nodes.

To update boundary nodes correctly, each process stores additional halo nodes containing copies of neighboring values. At every iteration, boundary information is exchanged between adjacent processes, ensuring consistency across the global solution.

Domain decomposition allows both the computational workload and memory requirements to be distributed among multiple processors, making it an essential technique for large-scale scientific simulations.

4 Mathematical Formulation

In order to solve the problem described in Section 2, these formulations are used.

The computational domain is discretized into N control volumes with uniform spacing

$$\Delta x = \frac{L}{N+1}.$$

Integrating the governing equation over a control volume and applying the finite volume method yields

$$a_P T_P = a_W T_W + a_E T_E.$$

The coefficients are computed as

$$a_W = \frac{\lambda_{i-\frac{1}{2}}}{\Delta x},$$

$$a_E = \frac{\lambda_{i+\frac{1}{2}}}{\Delta x},$$

and

$$a_P = a_W + a_E.$$

The interface conductivities are obtained through harmonic averaging.

The resulting algebraic system can be written in matrix form as

$$A\mathbf{T} = \mathbf{b},$$

where A is a sparse tridiagonal matrix and $\mathbf{T}$ contains the unknown temperatures.

5 Numerical Methodology

The numerical solution of the heat conduction problem consists of transforming the governing differential equation into a system of algebraic equations and solving it iteratively.

The overall procedure implemented in the code can be summarized as follows:

1. Generate the computational mesh.
2. Assign thermal conductivities to all nodes.
3. Compute the discretization coefficients.
4. Initialize the temperature field.
5. Perform halo communication between neighboring processes.
6. Execute Jacobi iterations.
7. Check convergence.
8. Gather the final solution and output results.

5.1 Mesh Generation

The spatial domain is discretized into a predefined number of nodes. The mesh generation routine computes both the local coordinates associated with each MPI process and the corresponding thermal conductivity values.

The interface position between materials is used to determine the conductivity assigned to each node.

5.2 Initialization

The temperature field is initialized before starting the iterative process. Boundary temperatures are imposed directly according to the problem statement, while internal nodes are initialized with an arbitrary value, typically obtained through a simple interpolation between boundary conditions.

5.3 Jacobi Iteration

At each iteration, every internal node is updated according to

$$T_i^{(k+1)} = \frac{a_W T_{i-1}^{(k)} + a_E T_{i+1}^{(k)} + b_i}{a_P}.$$

Because all updates use values from the previous iteration, the Jacobi method naturally supports parallel execution.

5.4 Convergence Criterion

The iterative process continues until the maximum difference between two consecutive temperature fields satisfies

$$\max_i \left| T_i^{(k+1)} - T_i^{(k)} \right| < 10^{-5}.$$

Each process computes its local error, and a global reduction operation is performed using MPI to determine the maximum error across all processes.

6 Parallel Implementation

The parallelization strategy adopted in this project is based on domain decomposition using MPI. The computational domain is divided into several subdomains, each assigned to a different MPI process.

6.1 Work Distribution

The total number of nodes is distributed among the available processes using a load-balancing routine. If the number of nodes is not perfectly divisible by the number of processes, the remaining nodes are distributed among the first processes to ensure balanced workloads.

Each process stores only its local portion of the mesh, reducing memory requirements and enabling scalability.

6.2 Local Data Structures

Each process stores (Figure 2):

- Local node coordinates.
- Local conductivity values.
- Local temperature field.
- Halo nodes.

The local temperature array contains

$$N_i + 2$$

elements, where the additional positions correspond to halo nodes.

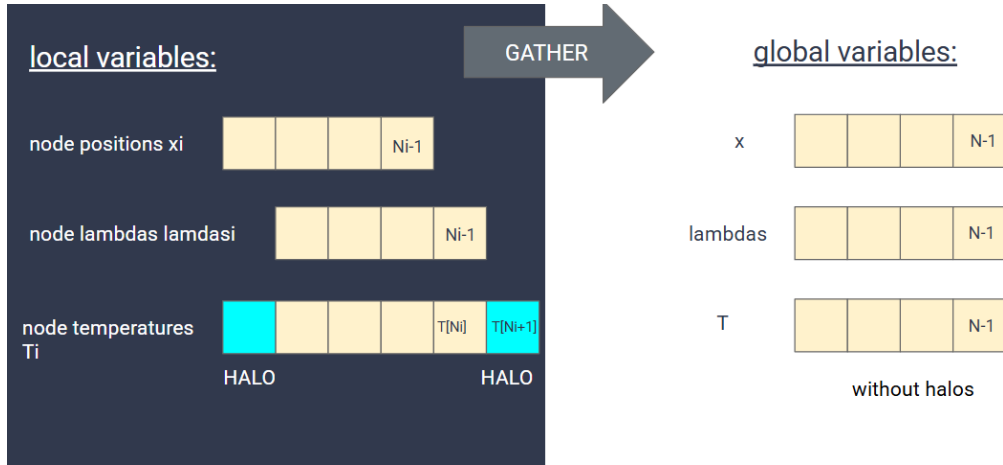


Figure 2: Memory allocation

6.3 Halo Nodes

Halo nodes are auxiliary values located at the boundaries of each subdomain. They contain copies of neighboring process values and are required to update boundary nodes during Jacobi iterations.

The local temperature vector is represented as

$$T[0], T[1], \dots, T[N_i], T[N_i + 1].$$

The elements $T[0]$ and $T[N_i + 1]$ correspond to halo nodes.

6.4 Halo Exchange

At every iteration, neighboring processes exchange boundary values using MPI communication routines.

The communication procedure consists of (Figure 3):

1. Sending the first internal node to the left neighbor.
2. Receiving the right halo value.
3. Sending the last internal node to the right neighbor.
4. Receiving the left halo value.

This operation ensures that all processes possess updated boundary information before performing the next Jacobi iteration.

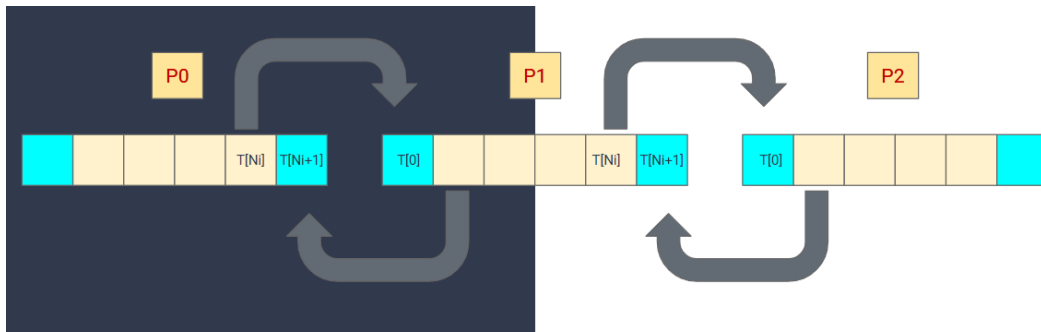


Figure 3: Halo update

6.5 Collective Communications

The implementation employs collective communication routines for global operations.

The routine

`MPI_Allreduce()`

is used to compute the global convergence error.

Similarly,

`MPI_Allgatherv()`

is used to reconstruct global vectors such as node coordinates and conductivity distributions.

7 Code Structure

The project follows a modular design philosophy in which each component is responsible for a specific task.

For compiling, use:

```
mpicc <main.c and all .c files in /sources/>
```

For running, use:

```
mpirun -np <NPROC> ./a.out
```

7.1 Main Program

The file `main.c` (Appendix A) controls the overall execution flow of the application.

Its responsibilities include:

- MPI initialization.
- Memory allocation.
- Mesh generation.
- Solver execution.
- Output generation.
- MPI finalization.

7.2 Mesh Generation Module

The file `mesh_generator.c` (Appendix B) contains routines for:

- Domain partitioning.
- Local mesh creation.
- Global mesh reconstruction.
- Conductivity assignment.

7.3 Pre-Solver Module

The file `pre_solver.c` (Appendix C) is responsible for:

- Halo communication.
- Harmonic averaging.
- Coefficient computation.

7.4 Jacobi Solver Module

The file `solver_jacobi.c` (Appendix D) implements:

- Jacobi iterations.
- Convergence evaluation.
- Matrix coefficient handling.

7.5 Output Module

The file `printing.c` (Appendix E) generates formatted output describing:

- Problem parameters.
- Numerical settings.
- Final results.

7.6 Testing Framework

The project includes two testing programs (Appendices F and G):

- `test_mesh.c`
- `test_jacobi.c`

These tests verify the correctness of mesh generation and solver implementation independently from the main program.

8 Verification and Testing

Verification is essential to ensure the correctness and reliability of numerical simulations. Several tests were conducted during development.

8.1 Mesh Verification

The mesh generation routines were validated by checking:

- Correct node distribution.
- Proper domain partitioning.
- Accurate conductivity assignment.

Particular attention was paid to the interface between materials, ensuring that conductivity transitions were handled correctly.

8.2 Jacobi Solver Verification

The Jacobi solver was tested independently using simplified systems with known analytical solutions.

The tests verified:

- Correct coefficient assembly.
- Proper iterative updates.
- Convergence behaviour.

8.3 Parallel Verification

The parallel implementation was validated by comparing solutions obtained using different numbers of MPI processes.

The final temperature distribution remained identical regardless of the process count, confirming the correctness of the communication strategy and domain decomposition.

9 Performance Analysis and Scalability

One of the primary objectives of this project was to evaluate the scalability of the MPI implementation. Execution times were measured for increasing numbers of processes while maintaining a fixed problem size.

9.1 Execution Time

The execution time decreases rapidly when moving from a single process to a moderate number of processes. The best performance was observed around 32 MPI processes (Figure 4). Beyond this point, the execution time begins to increase due to communication overhead.

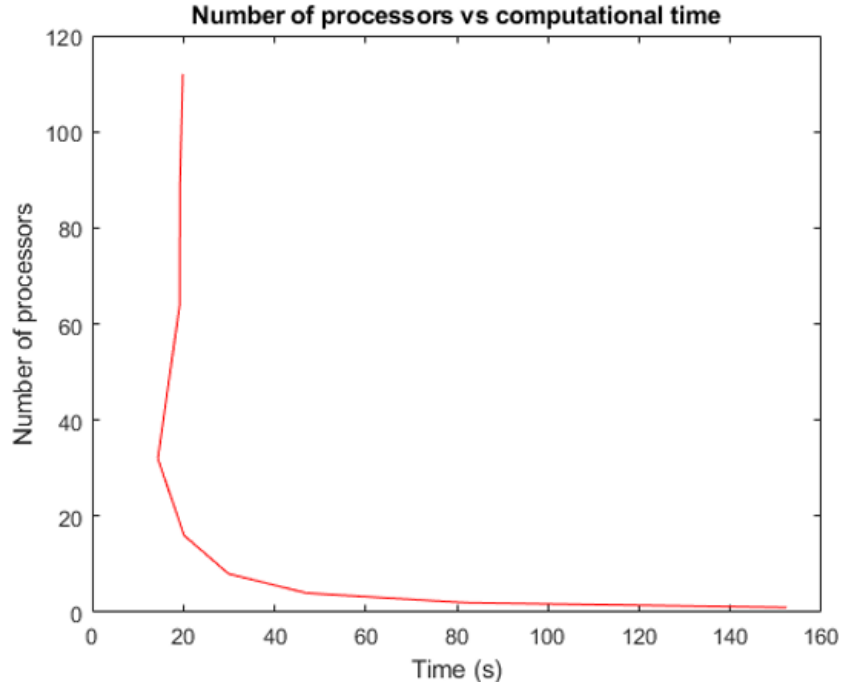


Figure 4: Number of Processors vs Computational Time

9.2 Speedup

The speedup is defined as

$$S(P) = \frac{T_1}{T_P},$$

where T_1 is the execution time using one process and T_P is the execution time using P processes.

Ideal scalability would produce

$$S(P) = P.$$

However, the measured speedup (Figure 5) deviates from the ideal line as the number of processes increases.

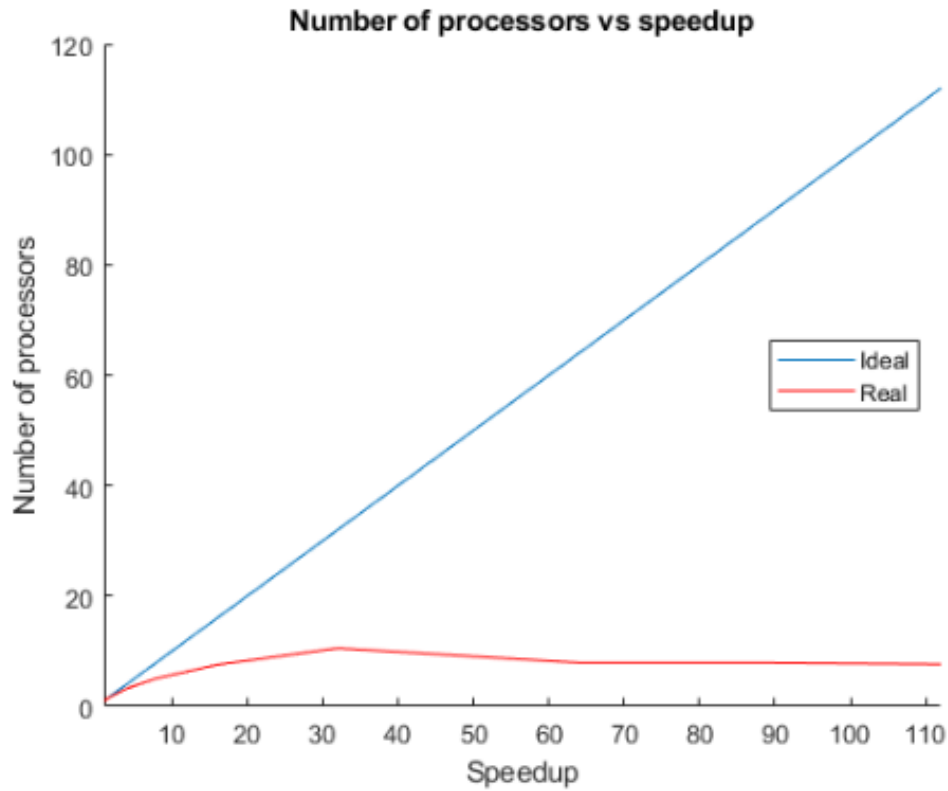


Figure 5: Number of processors vs speedup

9.3 Parallel Efficiency

Parallel efficiency is defined as

$$E(P) = \frac{S(P)}{P}.$$

Efficiency decreases progressively as more processes are added because communication costs become increasingly significant compared to the computational workload.

9.4 Communication Overhead

The main source of performance degradation is the halo exchange performed at every iteration.

As the number of processes increases:

- Local computation decreases.
- Communication frequency remains constant.
- Communication-to-computation ratio increases.

This effect ultimately limits scalability.

10 Conclusions

The objective of this project was to develop a parallel solver for the one-dimensional steady-state heat conduction equation in a composite wall using MPI. The governing equation was discretized using the Finite Volume Method, resulting in a sparse tridiagonal system that was solved iteratively through the Jacobi method.

The implementation successfully combined numerical methods and distributed-memory parallelism. Domain decomposition was employed to distribute the computational workload among multiple MPI processes, while halo-node communication ensured consistency between neighboring subdomains during the iterative solution process.

10.1 Numerical Results

The obtained temperature distribution was consistent with the expected physical behaviour of heat conduction through a composite wall. The implementation correctly handled the discontinuity in thermal conductivity at the material interface through harmonic averaging, preserving heat flux continuity across the domain.

Verification tests confirmed the correctness of:

- Mesh generation and domain decomposition.
- Thermal conductivity assignment.
- Jacobi iterative updates.
- Halo-node communications.
- Global convergence evaluation.

The numerical solution remained independent of the number of MPI processes employed, demonstrating the correctness of the parallel implementation.

10.2 Performance Analysis

The scalability study showed that parallelization significantly reduced execution times when compared to the sequential version. The largest performance gains were obtained when increasing the number of processes from small processor counts.

The execution time decreased rapidly up to approximately 32 MPI processes, where the best performance was observed. Beyond this point, the benefits of additional parallelism diminished and execution times began to increase.

This behaviour can be explained by the increasing impact of communication overhead. As the computational domain is divided among more processes, each process performs less computation while the amount of communication required for halo exchanges remains significant. Consequently, the communication-to-computation ratio increases, limiting scalability.

The speedup curves also deviated progressively from the ideal linear trend as the number of processes increased. This result is expected in distributed-memory applications where communication and synchronization costs become dominant at high processor counts.

10.3 Limitations

Although the Jacobi method is simple to implement and naturally parallelizable, its convergence rate is relatively slow compared with more advanced iterative methods. Furthermore, the use of blocking communication routines introduces additional synchronization overhead that can negatively affect performance at large process counts.

The present implementation is also restricted to a one-dimensional problem, which simplifies both the numerical discretization and communication patterns.

10.4 Final Remarks

Overall, the project successfully demonstrated the application of distributed-memory parallel computing techniques to the numerical solution of partial differential equations. The implementation provided valuable insight into domain decomposition, halo-node communication, iterative solvers, and scalability analysis, all of which constitute fundamental concepts in High Performance Computing. The results show that MPI-based parallelization can substantially improve computational performance, although communication overhead ultimately limits scalability at large processor counts.

11 Bibliography

References

- [Incropera et al., 2007] Incropera, F. P., DeWitt, D. P., Bergman, T. L., and Lavine, A. S. (2007). *Fundamentals of Heat and Mass Transfer*. John Wiley & Sons, Hoboken, NJ, 6 edition.
- [Message Passing Interface Forum, 2023] Message Passing Interface Forum (2023). MPI: A message-passing interface standard. Version 4.1.
- [Saad, 2003] Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, PA, 2 edition.
- [Versteeg and Malalasekera, 2007] Versteeg, H. K. and Malalasekera, W. (2007). *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. Pearson Education, 2 edition.

A main.c script

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "mpi.h"
4
5  #include "headers/mesh_generator.h"
6  #include "headers/pre_solver.h"
7  #include "headers/solver.h"
8  #include "headers/print.h"
9  #include "headers/tests.h"
10
11 //CHECK MPI STATUS
12 void checkr(int r,char *txt) {
13     if (r!=MPI_SUCCESS) {
14         fprintf(stderr,"Error: %s\n",txt);
15         exit(-1);
16     }
17 }
18
19 int main(int argc, char **argv) {
20     int myrank, mystart, myend, P, r, l; //variables for parallelizing
21
22     //PROBLEM KNOWN CONSTANTS
23     const int N = 1000; const double L = 5.0, dx = L/N, xchange = 2.8;
24     const double T0 = 100, TL = 200, lambda1 = 120, lambda2 = 180;
25
26     const double halo_left = -dx/2, halo_right = L + dx/2;
27     const double delta = 1e-5, T_ini = (T0+TL)*0.5; int Ni;
28
29     r=MPI_Init(&argc,&argv); checkr(r,"init");
30     r=MPI_Comm_rank(MPI_COMM_WORLD,&myrank); checkr(r,"rank");
31     r=MPI_Comm_size(MPI_COMM_WORLD,&P); checkr(r,"size");
32
33     //1.MESHING
34     worksplit(&mystart, &myend, myrank, P, N);
35     Ni = myend-mystart; //number of nodes for each process
36     double *x = (double*)malloc(N*sizeof(double)); //node positions
37     double *xi = (double*)malloc(Ni*sizeof(double)); //node positions for each process
38     double *lambdas = (double*)malloc(N*sizeof(double)); //lambdas for each node
39     double *lambdasi = (double*)malloc(Ni*sizeof(double)); //lambdas for each node of each process
40     generate_mesh(P, Ni, x, xi, dx, mystart, xchange, lambdasi, lambdas, lambda1, lambda2);
41
42
43     free (xi); free(lambdasi);
44
45     //2.INITIAL FIELD
46     double *Ti_s = (double*)malloc((Ni+2)*sizeof(double)); //supposed real node temperatures for each process
47     double *Ti_c = (double*)malloc((Ni+2)*sizeof(double)); //calculated real nodes temperatures for each process
48
49     for(int i=0;i<Ni+2;i++)
50         Ti_s[i]=T_ini; //Supposed temperatures
51
52     //3.BOUNDARY CONDITIONS
53     double b[2];
54
55     if(myrank==0)
56         Ti_s[0]=T0;
57     if(myrank==P-1)
58         Ti_s[Ni+1]=TL;
59
60     //pre iteration
61     double *a = (double*)malloc((3*Ni+4)*sizeof(double)); //a represents the 3 diagonals from tridiagonal
        coefficient matrix(from down to up)
62
63     double realdelta, global_delta, dub_time, time1, time2, elapsedg=0;
64     int iter = 1;

```

```

65
66 //4. ITERATION
67 time1=MPI_Wtime();
68 while(1){
69     //4.1. HALO UPDATE
70
71
72
73     halo_update(myrank, Ni, P, Ti_s);
74
75
76
77     b[0]=Ti_s[0];    b[Ni+1]=Ti_s[Ni+1];
78
79     //4.2. COEFFICIENTS
80     calc_coefficients(lambdas, lambda1, lambda2, a, Ni, P, mystart, myrank);
81
82     //5.2. JACOBI SOLVER
83
84
85     solver_Jacobi_tridiagonal(Ni+2, &realdelta, a, Ti_c, Ti_s, b);
86
87     for(int i=0; i<Ni+2; i++)
88         Ti_s[i]=Ti_c[i]; //update real nodes
89
90     //5.3. CONVERGENCE CRITERION
91
92
93     MPI_Allreduce(&realdelta, &global_delta, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
94     if(global_delta<delta)
95         break;
96
97     iter++;
98
99 }
100 time2=MPI_Wtime();
101
102 dub_time=time2-time1;
103 MPI_Reduce(&dub_time, &elapsedg, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
104
105
106
107
108
109 free(Ti_c); free(a);
110
111 //6. PRINTING
112
113 MPI_Barrier(MPI_COMM_WORLD);
114
115 double *T_global = NULL, *Ti_f;
116 Ti_f = malloc(Ni * sizeof(double));
117
118 if (myrank == 0)
119     T_global = malloc(N * sizeof(double));
120
121 for(int i=1; i<Ni+1; i++)
122     Ti_f[i-1]=Ti_s[i];
123
124 reunir_temperaturas(Ni, Ti_f, T_global, N, MPI_COMM_WORLD);
125
126
127 free(Ti_s); free(Ti_f);
128
129 if(myrank==0){
130     print_statement(L, xchange, lambda1, lambda2, T0, TL);
131     print_discretization(N, dx, x, lambdas, halo_left, halo_right);
132     print_initialf(T_ini);

```

```
133     print_bc(TL, T0);
134     print_iteration();
135     print_results(T_global, N, iter,delta);
136     free(T_global);
137 }
138
139 free(x); free(lambdas);
140
141 if(myrank==0)
142     printf("Time-----> %lf\n",dub_time);
143
144 MPI_Finalize();
145 /*
146 SERIAL TEST AREA
147 test_meshing(N,L,dx,lambda1,lambda2,xchange);
148 */
149 }
```

B mesh_generator.c script

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "mpi.h"
4  #include "../headers/mesh_generator.h"
5
6  void worksplit(int *mystart, int *myend, int myrank, int P, int N){
7      int op=N/P;
8      int shift=N-op*P;
9
10     if(P<N){
11         *mystart=myrank*op;
12         *myend=*mystart+op;
13         if(shift>myrank){
14             *mystart+=myrank;
15             *myend+=myrank+1;
16         }
17         else{
18             *mystart+=shift;
19             *myend+=shift;
20         }
21     }
22     else{
23         if(myrank<N){
24             *mystart =myrank;
25             *myend=myrank+1;
26         }
27         else{
28             *mystart=N;
29             *myend=N;
30         }
31     }
32 }
33
34 void generate_mesh(int P, int Ni, double *x, double *xi, double dx, int mystart, double xchange,
35                   double *lambdasi, double *lambdas, double lambda1, double lambda2){
36     //local mesh
37     for(int i=0; i<Ni; i++){
38         xi[i] = dx*(mystart+1+i)-dx/2;
39         if(xi[i]<xchange)
40             lambdasi[i]=lambda1;
41         else
42             lambdasi[i]=lambda2;
43     }
44
45     //global mesh
46     allgathering(Ni, P, x, xi);
47     allgathering(Ni, P, lambdas, lambdasi);
48 }
49
50 void allgathering(int send_count, int P, double *X, double *sendbuf){
51     //index adjustment
52     int *recvcounts=(int*)malloc(P*sizeof(int));
53     int *displs=(int*)malloc(P*sizeof(int));
54
55     MPI_Barrier(MPI_COMM_WORLD);
56     MPI_Allgather(&send_count, 1, MPI_INT, recvcounts, 1, MPI_INT, MPI_COMM_WORLD);
57     displs[0]=0;
58     for (int i=1; i<P; ++i){
59         displs[i]=displs[i-1]+recvcounts[i-1];
60     }
61
62     MPI_Barrier(MPI_COMM_WORLD);
63     MPI_Allgatherv(sendbuf, send_count, MPI_DOUBLE, X, recvcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD);
64
65     free(recvcounts); free(displs);}

```


C pre_solver.c script

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "mpi.h"
4  #include "../headers/pre_solver.h"
5
6  void halo_update(int myrank, int Ni, int Nproc, double *Ti_s) {
7
8      if (myrank > 0) {
9          // Send first node to left
10         MPI_Send(&Ti_s[1], 1, MPI_DOUBLE, myrank - 1, 0, MPI_COMM_WORLD);
11     }
12     if (myrank < Nproc - 1) {
13         // Left halo receives from right
14         MPI_Recv(&Ti_s[Ni+1], 1, MPI_DOUBLE, myrank + 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
15     }
16
17     if (myrank < Nproc - 1) {
18         // Send last node to right
19         MPI_Send(&Ti_s[Ni], 1, MPI_DOUBLE, myrank + 1, 1, MPI_COMM_WORLD);
20     }
21     if (myrank > 0) {
22         // Right halo receives from left
23         MPI_Recv(&Ti_s[0], 1, MPI_DOUBLE, myrank - 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
24     }
25 }
26
27
28 double lambdaharm(double lambdai, double lambdaj){
29     return ((2*lambdai*lambdaj)/(lambdai+lambdaj));
30 }
31
32 void calc_coefficients(double *lambdas, double lambda1, double lambda2, double *a, int Ni, int P,
33                       int mystart, int myrank){
34
35     double* lambdaE = (double*)malloc(Ni*sizeof(double));
36     double* lambdaW = (double*)malloc(Ni*sizeof(double));
37     double* lambdaP = (double*)malloc(Ni*sizeof(double));
38
39     for(int i=0; i<Ni; i++){
40         lambdaW[i] = (myrank==0 && (i==0)) ? lambda1 : lambdaharm(lambdas[i+mystart], lambdas[i+mystart-1]);
41         lambdaE[i] = (myrank==P-1 && (i==Ni-1)) ? lambda2 : lambdaharm(lambdas[i+mystart], lambdas[i+mystart+1]);
42         lambdaP[i] = (lambdaW[i] + lambdaE[i]);
43     }
44
45     int idx = 0;
46
47     for(int i = 0; i < Ni; i++)
48         a[idx++] = -lambdaW[i];
49
50     a[idx++] = 0;
51     a[idx++] = 1;
52
53     for(int i = 0; i < Ni; i++)
54         a[idx++] = lambdaP[i];
55
56     a[idx++] = 1;
57     a[idx++] = 0;
58
59     for(int i = 0; i < Ni; i++)
60         a[idx++] = -lambdaE[i];
61
62     free(lambdaE); free(lambdaW); free(lambdaP);
63 }

```

D solver_jacobi.c script

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include "mpi.h"
5  #include "../headers/solver.h"
6
7  #define AC_MAT(A, n, i, j) A[(i)*(n)+(j)]
8
9
10 void get_Diagonal_Inv_Square(double *Dinv, double *A, int m) {
11     for (int i = 0; i < m; i++)
12         Dinv[i] = 1.0 / A[m - 1 + i]; // diagonal central
13
14 }
15
16 void get_Negative_Diagonal_Inf_Square(double *L, double *A, int m) {
17     for (int i = 0; i < m - 1; i++)
18         L[i] = -A[i]; // diagonal inferior
19 }
20
21 void get_Negative_Diagonal_Sup_Square(double *U, double *A, int m) {
22     for (int i = 0; i < m - 1; i++)
23         U[i] = -A[2 * m - 1 + i]; // diagonal superior
24 }
25
26 }
27
28 void Diagonal_Mult(double *Dinv, double *L, double *U, int m, double *T) {
29     for (int i = 0; i < m - 1; i++) {
30         AC_MAT(T, m - 1, 0, i) = Dinv[i] * U[i]; // upper
31         AC_MAT(T, m - 1, 1, i) = Dinv[i + 1] * L[i]; // lower
32     }
33 }
34
35 }
36
37 void SolveEquationforx(double *T, double *C, double *xk, double *x, int m, double *realdelta) {
38     double error = 0.0;
39
40     x[0] = AC_MAT(T, m - 1, 0, 0) * xk[1] + C[0];
41     error = fabs(x[0] - xk[0]);
42
43     x[m - 1] = AC_MAT(T, m - 1, 1, m - 2) * xk[m - 2] + C[1];
44     if (fabs(x[m - 1] - xk[m - 1]) > error)
45         error = fabs(x[m - 1] - xk[m - 1]);
46
47     for (int i = 1; i < m - 1; i++) {
48         x[i] = AC_MAT(T, m - 1, 0, i) * xk[i + 1] + AC_MAT(T, m - 1, 1, i - 1) * xk[i - 1];
49         //printf("\n %lf = %lf * %lf + %lf * %lf", x[i], AC_MAT(T, m - 1, 0, i), xk[i + 1], AC_MAT(T, m - 1, 1, i - 1), xk[i - 1]);
50         //fflush(stdout);
51         double local_err = fabs(x[i] - xk[i]);
52         if (local_err > error)
53             error = local_err;
54     }
55
56     *realdelta = error;
57 }
58
59 void solver_Jacobi_tridiagonal(int m, double *realdelta, double *A, double *x, double *xk, double *b) {
60     double *Dinv = (double *)malloc(m * sizeof(double));
61     double *L = (double *)malloc((m - 1) * sizeof(double));
62     double *U = (double *)malloc((m - 1) * sizeof(double));
63     double *T = (double *)malloc(2 * (m - 1) * sizeof(double));
64     double C[2];

```

```
65     int iter = 0;
66
67     get_Diagonal_Inv_Square(Dinv, A, m);
68     get_Negative_Diagonal_Inf_Square(L, A, m);
69     get_Negative_Diagonal_Sup_Square(U, A, m);
70     Diagonal_Mult(Dinv, L, U, m, T);
71
72     C[0] = Dinv[0] * b[0];
73     C[1] = Dinv[m - 1] * b[1];
74
75     SolveEquationforx(T, C, xk, x, m, realdelta);
76
77
78     free(Dinv); free(L); free(U); free(T);
79 }
```

E printing.c script

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include "../headers/print.h"
4
5  void print_statement(double L, double xchange, double lambda1,
6                      double lambda2, double T0, double TL){
7      printf("1D HEAT CONDUCTION PROBLEM IN A COMPOSITE WALL\n\n"
8             "PROBLEM STATEMENT\n\n");
9      printf("A wall of total length %fm is composed of "
10             "two materials joined at x_c = %fm. The first segment "
11             "(0<=x<x_c) is made of a material A with thermal conductivity "
12             "lambda_1 = %fW/mK, and the second segment (x_c<=x<L) is made "
13             "of material B with thermal conductivity lambda_1 = %fW/mK.\n\n"
14             "The system is in steady-state, and heat conduction is "
15             "one-dimensional along the x-axis. The temperature at "
16             "the left end of the wall is maintained at T_0 = %fK, and the "
17             "temperature at the right end is maintained at T_L = %fK.\n\n",
18             L,xchange,lambda1,lambda2,T0,TL);
19      printf("Assuming:\n-Steady-state heat conduction\n-No internal heat "
20             "generation\n-Perfect thermal contact between the two materials "
21             "\n-Constant thermal conductivity in each material\n-One-dimensional "
22             "conduction only.\n\nDetermine the temperature distribution T(x) along "
23             "the lenght of the wall.\n");
24  }
25
26  void print_discretization(int N, double dx, double* x, double *lambdas, double halo_left, double halo_right){
27      printf("\n\n1. DISCRETIZATION OF THE MESH\n\n");
28      printf("Contemplating %d internal nodes:\ndx = %f \nNode positions [m] = {" ,N,dx);
29      for(int i=0;i<N;i++){
30          printf("%f",x[i]);
31          if(i<(N-1))
32              printf(", ");
33      }
34      printf("}\nThermal conductivities per node [W/mK] = {" );
35      for(int i=0;i<N;i++){
36          printf("%f",lambdas[i]);
37          if(i<(N-1))
38              printf(", ");
39      }
40      printf("}\n\nTwo external nodes (HALO) are also considered at x[0] = "
41             "%fm and x[N+1] = %fm.\n", halo_left, halo_right);
42  }
43
44  void print_initialf(double Tini){
45      printf("\n\n2. INITIAL MAP DEFINITION\n\n");
46      printf("T_supposed[i]=T_initial=%fK\n",Tini);
47  }
48
49  void print_bc(double TL, double T0){
50      printf("\n\n3. BOUNDARY CONDITIONS\n\n");
51      printf("T[0]=T_0=%fK\n",T0);
52      printf("T[N+1]=T_L=%fK\n",TL);
53  }
54
55  void print_iteration(){
56      printf("\n\n4. ITERATION\n\n");
57      printf("    4.1. Halo update.\n    4.2. Calculation of coefficients (aP,aW,aE,b). "
58             "\n    4.3. Map resolution by the Jacobi method.\n"
59             "    4.4. Is max|T_calculated[i] - T_supposed[i]|<delta?\n        NO: "
60             "T_supposed[i] = T_calculated[i] and back to point 4.1.\n        YES: end of the iteration.");
61  }
62
63  void print_results(double *T, int N, int iter, double delta){
64      printf("\n\n6. RESULTS\n\n");
65      printf("Contemplating delta = %lf, and with %d iterations:\n",delta,iter);

```

```
66     printf("T[i] [K] (internal nodes)= {");
67     for(int i=0;i<N;i++){
68         printf("%f",T[i]);
69         if(i<(N-1))
70             printf(", ");
71     }
72     printf("}\n");
73 }
```

F test_mesh.c script

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include "../headers/tests.h"
4
5 void test_meshing(int N, double L, double dx, double lambda1, double lambda2, double xchange){
6     double *x = (double*)malloc((N+2)*sizeof(double));
7     double *x_cv = (double*)malloc((N+2)*sizeof(double));
8     double *lambda = (double*)malloc((N+2)*sizeof(double));
9
10    printf("X CONTROL VOLUMES (TEST): ");
11    for(int i=0; i<N+2; i++){
12        x_cv[i] = dx*i;
13        printf("%f ", x_cv[i]);
14    }
15    printf("\n");
16
17    printf("X NODES (TEST): ");
18    for(int i=0; i<(N+2); i++){
19        x[i] = -dx/2+i*dx;
20        printf("%f ", x[i]);
21    }
22    printf("\n");
23
24    printf("LAMBDA X (TEST): ");
25    for(int i=0; i<(N+2); i++){
26        if(x[i]<xchange)
27            lambda[i]=lambda1;
28        else
29            lambda[i]=lambda2;
30        printf("%f ", lambda[i]);
31    }
32    printf("\n");
33
34    free(x); free(x_cv); free(lambda);
35 }
```

G test_jacobi.c script

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include "../headers/tests.h"
4
5 void test_meshing(int N, double L, double dx, double lambda1, double lambda2, double xchange){
6     double *x = (double*)malloc((N+2)*sizeof(double));
7     double *x_cv = (double*)malloc((N+2)*sizeof(double));
8     double *lambda = (double*)malloc((N+2)*sizeof(double));
9
10    printf("X CONTROL VOLUMES (TEST): ");
11    for(int i=0; i<N+2; i++){
12        x_cv[i] = dx*i;
13        printf("%f ", x_cv[i]);
14    }
15    printf("\n");
16
17    printf("X NODES (TEST): ");
18    for(int i=0; i<(N+2); i++){
19        x[i] = -dx/2+i*dx;
20        printf("%f ", x[i]);
21    }
22    printf("\n");
23
24    printf("LAMBDA X (TEST): ");
25    for(int i=0; i<(N+2); i++){
26        if(x[i]<xchange)
27            lambda[i]=lambda1;
28        else
29            lambda[i]=lambda2;
30        printf("%f ", lambda[i]);
31    }
32    printf("\n");
33
34    free(x); free(x_cv); free(lambda);
35 }
```

H Headers files used for the project

mesh_generator.h

```

1  #ifndef MESH_GENERATOR_H
2  #define MESH_GENERATOR_H
3
4  void worksplit(int *mystart, int *myend, int myrank, int P, int N);
5  void generate_mesh(int P, int Ni, double *x, double *xi, double dx, int mystart, double xchange,
6    double *lambdasi, double *lambdas, double lambda1, double lambda2);
7  void allgathering(int send_count, int P, double *X, double *sendbuf);
8
9  #endif

```

pre_solver.h

```

1  #ifndef PRE_SOLVER_H
2  #define PRE_SOLVER_H
3
4  void halo_update(int myrank, int Ni, int Nproc, double *Ti_c);
5  double lambdaharm(double lambda1, double lambda2);
6  void calc_coefficients(double *lambdas, double lambda1, double lambda2, double *a, int Ni, int P,
7    int mystart, int myrank);
8
9  #endif

```

print-h

```

1  #ifndef PRINT_H
2  #define PRINT_H
3
4  void print_statement(double L, double xchange, double lambda1,
5    double lambda2, double T0, double TL);
6  void print_discretization(int N, double dx, double* x, double *lambdas, double halo_left, double halo_right);
7  void print_bc(double TL, double T0);
8  void print_initialf(double Tini);
9  void print_iteration();
10 void print_results(double *T, int N, int iter, double delta);
11
12 #endif

```

solver.h

```

1  #ifndef SOLVER_H
2  #define SOLVER_H
3
4  void get_Diagonal_Inv_Square(double *Dinv, double *A, int m);
5  void get_Negative_Diagonal_Inf_Square(double *L, double *A, int m);
6  void get_Negative_Diagonal_Sup_Square(double *U, double *A, int m);
7  void Diagonal_Mult(double *Dinv, double *L, double *U, int m, double *T);
8  void SolveEquationforx(double *T, double *C, double *xk, double *x, int m, double *realdelta);
9  void solver_Jacobi_tridiagonal(int m, double *realdelta, double *A, double *x, double *xk, double *b);
10
11 #endif

```

tests.h

```

1  #ifndef SOLVER_H
2  #define SOLVER_H
3
4  void get_Diagonal_Inv_Square(double *Dinv, double *A, int m);
5  void get_Negative_Diagonal_Inf_Square(double *L, double *A, int m);
6  void get_Negative_Diagonal_Sup_Square(double *U, double *A, int m);
7  void Diagonal_Mult(double *Dinv, double *L, double *U, int m, double *T);
8  void SolveEquationforx(double *T, double *C, double *xk, double *x, int m, double *realdelta);
9  void solver_Jacobi_tridiagonal(int m, double *realdelta, double *A, double *x, double *xk, double *b);
10
11 #endif

```